Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM302 Үйлдлийн систем

Бие даалт №1. Banker's Algorithm

Шалгасан багш: Г.Ганбат /F.CS10/
Гүйцэтгэсэн оюутан: Ц.Цэндсүрэн /B242270128/
/B231940023/
/B241910004/
/B241940034/
/B242270041/

Улаанбаатар хот
2026 он

Хураангуй

Асуудал. Түгжрэл (deadlock) нь олон процесс хязгаарлагдмал нөөцийг өрсөлдөн авах үед үүсдэг үйлдлийн системийн сонгодог асуудал. Banker's Algorithm нь нөөц олгохын өмнө 'олгосны дараа бүх процесс эцэст нь ажлаа дуусгаж чадах уу?' гэж шалгаж, системийг үргэлж SAFE төлөвт байлгах замаар түгжрэлээс сэргийлдэг.

Шийдэл. Safety Algorithm, Resource-Request Algorithm, Deadlock Detection алгоритмуудыг Java хэл дээр хэрэгжүүлж, алгоритмын ажиллагааг алхам бүрээр нь Resource-allocation graph (процесс ба нөөцийн граф), Allocation/Max/Need матриц, псевдокодын мөр тодруулалт, үг хэллэгээр тайлбарласан Swing симуляторт холбосон. Хөдөлгүүр нь туршилтыг урьдчилан бүх алхамд задалж (snapshot) хадгалдаг тул хэрэглэгч урагш, хойш алхаж, тоглуулж, хурдыг тохируулж болно.

Туршилт. 10 бэлэн туршилт (сурах бичгийн жишээ ба дасгал 8.3, 8.9, 8.28 зэрэг), 29 автомат тест, мөн 3000 санамсаргүй төлөвийг брут-форс алгоритмтай харьцуулсан fuzz шалгалт хийсэн.

Гол үр дүн. 29/29 тест тэнцсэн. Fuzz шалгалтад Safety Algorithm-ийн дүгнэлт брут-форсоос нэг ч удаа зөрөөгүй (зөрүү = 0), зөвшөөрөгдсөн 716 хүсэлт бүрийн дараа систем SAFE хэвээр байсан. Banker асаалттай үед 'нөөц байгаа ч АЮУЛТАЙ тул татгалзах' тохиолдлыг, унтраалттай үед бодит түгжрэл үүсэхийг симулятор нэг систем дээр харьцуулан харуулж чадсан.

Хязгаарлалт (товч). Хэрэглэгч Max/Allocation/Available болон хүсэлтийг UI дээрээс гараар оруулах боломж одоогоор байхгүй: туршилтууд `ScenarioLibrary.java`-д тодорхойлогдсон. Дэлгэрэнгүйг 9-р бүлэгт тусгав.

1. Удиртгал

1.1 Зорилго

Энэхүү төслийн зорилго нь сурах бичгийн 8-р бүлгийн Programming Project болох Banker's Algorithm-ийг бүрэн хэрэгжүүлж, түүний ажиллагааг хэрэглэгч алхам алхмаар ажиглаж, өөр нөхцөлөөр дахин туршиж болохуйц интерактив дүрслэлтэй болгох явдал юм.

1.2 Ач холбогдол

Түгжрэлээс сэргийлэх алгоритмыг зөвхөн хүснэгт дээр гараар бодохоос гадна 'яагаад энэ хүсэлтийг өгөхгүй байна вэ?' гэдгийг процесс ба нөөцийн графаар харах нь ойлголтыг эрс хөнгөвчилдөг. Симулятор нь SAFE / UNSAFE / DEADLOCK гэсэн гурван ойлголтын ялгааг (UNSAFE нь заавал DEADLOCK биш) бодит жишээгээр харуулдаг.

1.3 Хамрах хүрээ

Safety Algorithm, Resource-Request Algorithm, Deadlock Detection, 10 туршилт, алхам алхмаар тоглуулах UI, автомат тест.

2. Сурах бичгийн шаардлага

Сурах бичгийн 8-р бүлгийн төсөлд банкир олон 'үйлчлүүлэгч'-д (процесс) хязгаарлагдмал нөөцийг олгохдоо Banker's Algorithm-ийг ашиглан зөвхөн системийг аюулгүй төлөвт үлдээх хүсэлтийг зөвшөөрөх ёстой. Үндсэн шаардлагыг дараах байдлаар ойлгож, хэрэгжилттэй харьцуулав:

№	Шаардлага	Хэрэгжилт	Төлөв
1	Available, Max, Allocation, Need өгөгдлийн бүтэц	`SystemState` (Need = Max − Allocation автоматаар)	Хэрэгжсэн
2	Хүсэлт шалгах: Request ≤ Need, Request ≤ Available	`BankerAlgorithm.requestResources`, `SimulationEngine.processRequest`	Хэрэгжсэн
3	Түр олгож Safety Algorithm-аар шалгах; UNSAFE бол буцаах	түр хуулбар (`copy()`) дээр allocate → checkSafety → зөвшөөрөх/татгалзах	Хэрэгжсэн
4	Safe sequence гаргах	`checkSafety(state, seq)`	Хэрэгжсэн
5	Нөөц чөлөөлөх (release)	Зөвхөн процесс дуусахад бүх нөөцөө буцаана (`SystemState.finish`). Тусдаа release хүсэлт байхгүй	Хэрэгжсэн

3. Онолын үндэс

3.1 Нөөц ба түгжрэл

Түгжрэл нь дөрвөн нөхцөл (mutual exclusion, hold and wait, no preemption, circular wait) нэгэн зэрэг биелэхэд үүснэ. Banker's Algorithm нь *deadlock avoidance* буюу нөөцийг олгохоос өмнө ирээдүйн төлвийг урьдчилан шалгах арга бөгөөд процесс бүр нийт хэрэгцээгээ (Max) урьдчилан зарласан байхыг шаарддаг. Энэ алгоритмыг анх Э. Дейкстра санал болгосон.

3.2 Өгөгдлийн бүтэц

n процесс, m төрлийн нөөцтэй системд:

Нэр	Хэмжээс	Утга
Available	m	Системд яг одоо чөлөөтэй байгаа нөөц
Max	$n \times m$	Процесс бүрийн зарласан дээд хэрэгцээ
Allocation	$n \times m$	Процесс бүр яг одоо барьж буй нөөц
Need	$n \times m$	Need = Max − Allocation: дахин авах ёстой нөөц
Work, Finish	m, n	Safety Algorithm-ийн түр вектор ба 'дуусаж чадна' тэмдэг

3.3 Safety Algorithm

1. Work = Available; Finish[i] = false (бүх i)
2. Finish[i] == false БА Need[i] ≤ Work байх i-г ол
3. Олдвол: Work = Work + Allocation[i]; Finish[i] = true; 2-р алхам руу буц
4. Олдохгүй бол 5-р алхам руу
5. Бүх Finish[i] == true бол систем SAFE, үгүй бол UNSAFE

Хугацааны нарийвчлал нь $O(m \cdot n^2)$: дээд тал нь n удаа давталт, давталт бүрд n процесс тус бүрийг m нөөцөөр харьцуулна.

3.4 Resource-Request Algorithm

P_i нь Request[i] хүснэ:

1. Request[i] $\leq$ Need[i] мөн үү? үгүй бол $\rightarrow$ АЛДАА (хэт их хүсэлт)
2. Request[i] $\leq$ Available мөн үү? үгүй бол $\rightarrow$ P_i ХҮЛЭЭНЭ
3. Түр олго: Available $-=$ Request; Allocation[i] $+=$ Request; Need[i] $-=$ Request
4. Safety Algorithm-аар шалга
5. SAFE бол нөөцийг ОЛГО
6. UNSAFE бол түр олгосныг БУЦААЖ, P_i ХҮЛЭЭНЭ

3.5 Deadlock Detection

Банкиргүй (Banker унтраалттай) системд түгжрэл үүсэх боломжтой. Detection алгоритм нь Safety Algorithm-тэй бараг ижил бөгөөд Need-ийн оронд процессын одоогийн Request-ийг Work-тэй харьцуулна. Төгсгөлд Finish[i] = false үлдсэн процессууд түгжрэлд орсон байна. Энэ хэрэгжилтэд Request вектор нь Need-тэй тэнцүү гэж авсан (процесс бүр үлдсэн хэрэгцээгээ бүгдийг нэг дор хүсэх хамгийн муу тохиолдол).

3.6 SAFE, UNSAFE, DEADLOCK-ийн ялгаа

SAFE төлөвт түгжрэл гарахгүй. UNSAFE төлөвт түгжрэл гарах эрсдэл байгаа боловч заавал гарах албагүй. DEADLOCK нь процессууд бодитоор бие биеэ хүлээж гацсан төлөв. Banker's Algorithm нь системийг UNSAFE төлөвт оруулах хүсэлтийг татгалзаж, тиймээс DEADLOCK-д хэзээ ч хүргэхгүй.

3.7 Гараар бодсон сурах бичгийн жишээ

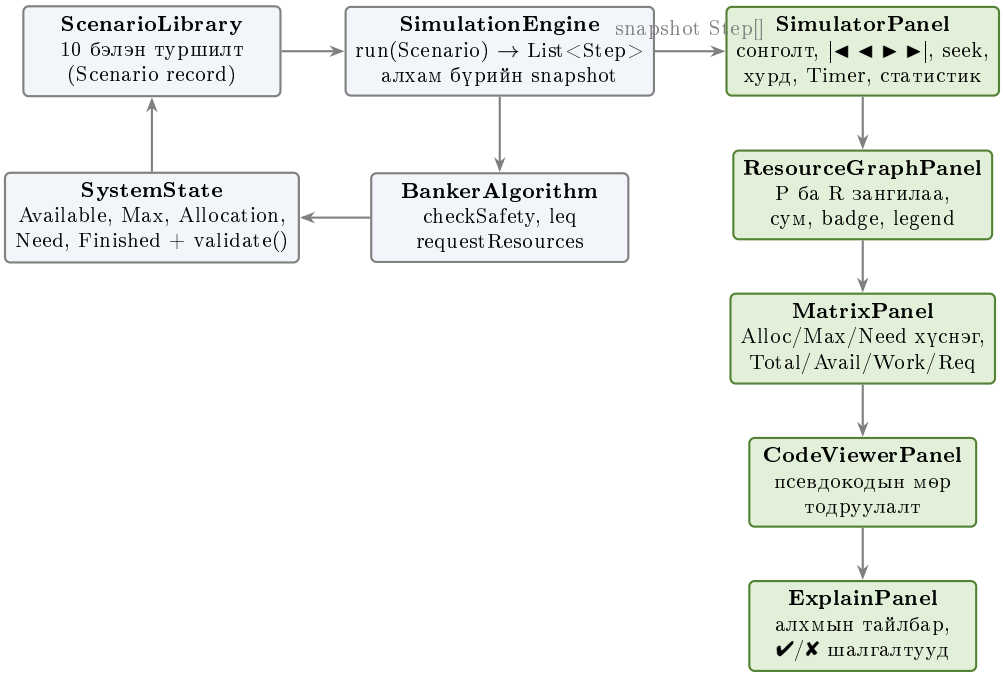
5 процесс, R0=10, R1=5, R2=7. Эхний Available = (3,3,2). Need = Max − Allocation: P0=(7,4,3), P1=(1,2,2), P2=(6,0,0), P3=(0,1,1), P4=(4,3,1).

Алхам	Сонгосон процесс	Шалгалт Need $\leq$ Work	Work (шинэ)
0	—	—	(3,3,2)
1	P1	(1,2,2) $\leq$ (3,3,2) ✔	(3,3,2)+(2,0,0) = (5,3,2)
2	P3	(0,1,1) $\leq$ (5,3,2) ✔	(5,3,2)+(2,1,1) = (7,4,3)
3	P0	(7,4,3) $\leq$ (7,4,3) ✔	(7,4,3)+(0,1,0) = (7,5,3)
4	P2	(6,0,0) $\leq$ (7,5,3) ✔	(7,5,3)+(3,0,2) = (10,5,5)
5	P4	(4,3,1) $\leq$ (10,5,5) ✔	(10,5,5)+(0,0,2) = (10,5,7)

Дараалал ⟨P1, P3, P0, P2, P4⟩ нь SAFE. Сурах бичигт ⟨P1, P3, P4, P0, P2⟩ дарааллыг жишээ болгодог. Хоёулаа хүчинтэй Safe sequence: хэрэгжилт нь индексийн өсөх дарааллаар эхний тохирохыг сонгодог тул өөр дараалал гаргадаг (тест T1b, T1c-д хоёуланг нь бие даасан шалгагчаар баталгаажуулсан).

4. Системийн зохиомж

Систем нь алгоритмын тооцоолол ба дүрслэлийг салгасан. `SimulationEngine` нь туршилтыг нэг удаа бүтнээр нь ажиллуулж, алхам бүрийн төлвийн snapshot-ыг (`Step`) жагсаалтад хадгална. UI нь зөвхөн тухайн индексийн `Step`-ийг зурдаг. Иймээс хойш алхах, шууд харайх (seek) нь тооцоолол дахин хийлгүй ажиллана.



Зураг 4.1. Компонент ба өгөгдлийн урсгал (ногоон нь UI)

Класс	Үүрэг	Мөр
`BankerAlgorithm`	`leq`, `checkSafety`, `requestResources` (цэвэр алгоритм)	85
`SystemState`	Төлөв, Need тооцоолол, баталгаажуулалт, хуулбар, allocate/finish	107
`Scenario`, `ScenarioLibrary`	туршилтйн record ба 10 бэлэн туршилт	30 / 116
`SimulationEngine`	туршилтыг ажиллуулж Step жагсаалт үүсгэх, тайлбар текст	497
`Step`	Алхам бүрийн snapshot (матриц, статус, Work, Finish, тоолуур)	40
`SimulatorPanel`	Удирдлага (сонголт, ▶, slider), панелуудыг холбох	210
`ResourceGraphPanel`	Resource-allocation graph зурах	293
`MatrixPanel`, `CodeViewerPanel`, `ExplainPanel`	Матриц, псевдокод, тайлбар	159 / 99 / 54

Нийт эх код 13 файл, ойролцоогоор 1715 мөр (`src/main/java/com/os`). Тест ба скрипт тусдаа (`tests/`).

5. Хэрэгжилт

5.1 Хэл ба орчин

Java (`record` ашигласан тул JDK 16+, баримтжуулалтад JDK 17+ гэж заасан), Swing/AWT. Гадаад сан ашиглаагүй. Build систем (Maven/Gradle) ашиглаагүй: `javac` командаар шууд compile хийнэ.

5.2 Гол шийдлүүд

- **Төлөвийн хамгаалалт.** `SystemState` нь getter бүрд хуулбар буцааж (`getAvailable()` гэх мэт), гадны код төлвийг санамсаргүй өөрчлөхөөс хамгаална. Конструктор нь сөрөг утга, Allocation > Max, хэмжээс таарахгүй байхыг `IllegalArgumentException`-ээр татгалзана.
- **Түр олголт.** Хүсэлтийг `state.copy()` дээр туршиж, SAFE бол `copyFrom`-оор шилжүүлнэ; UNSAFE бол анхны төлөв огт өөрчлөгдөхгүй (тест T4b).
- **Snapshot загвар.** `Step` бүр матриц, статус, Work, Finish, safe sequence, хүлээгдэж буй хүсэлт, тоолуурыг хуулбарлан хадгална. Үүгээр UI нь төлөвгүй (stateless) зурна.
- **Хоёр үе шат.** (1) туршилтйн хүсэлтүүдийг дарааллаар шийднэ; (2) үлдсэн процессуудыг Safe sequence-ийн дагуу ажиллуулж дуусгана. Банкиргүй горимд дараа нь Detection ажиллуулж түгжрэлийг илрүүлнэ.
- **Хүлээлгийн дараалал.** Нөөц хүрэлцээгүй эсвэл UNSAFE байсан хүсэлт `waiting` жагсаалтад орж, процесс дуусан нөөц чөлөөлөгдөх бүрд дахин шалгагдана (`retryWaiting`).

5.3 Жишээ код: Safety Algorithm

```
public static boolean checkSafety(SystemState state, List<Integer> safeSequence) {
        int[] work = state.getAvailable();   int[][] need = state.getNeed();
        int[][] allocation = state.getAllocation();
        boolean[] finish = state.getFinished();
        safeSequence.clear();
        while (true) {
                int found = -1;
                for (int p = 0; p < n; p++)
                if (!finish[p] && leq(need[p], work)) { found = p; break; }
                if (found < 0) break;
                for (int k = 0; k < m; k++) work[k] += allocation[found][k];
                finish[found] = true;  safeSequence.add(found);
        }
        for (boolean f : finish) if (!f) return false;   // UNSAFE
        return true;                                      // SAFE
}
```

5.4 Бэлэн код, сан, AI-ийн хэрэглээ

Гадаад сан ашиглаагүй (зөвхөн JDK-ийн Swing/AWT). Алгоритмын псевдокод нь сурах бичгийн 8-р бүлэгт үндэслэсэн. AI: баримт бичиг (Report, Manual, README), автомат тест, build скриптийг бэлтгэхэд Claude ашиглан туслалцаа авч өөрөө хийсэн. Алгоритмын кодыг сурах бичгийн жишээг авч gemini, chatgpt ашиглан өөрийн логикт тохируулан бичиж нэмэлт засвар логикийг өөрөө сайжруулсан.

6. Туршилт ба UI дүрслэл

6.1 Юуг өөрчилж турших вэ

- **туршилт сонгох** (дээд талын жагсаалт, 10 сонголт): өөр Available, Max, Allocation, хүсэлтийн дараалал, Banker асаалттай/унтраалттай горим.
- **Алхам удирдах:** |◄ эхлэл, ◄ өмнөх, ► дараах, ►| төгсгөл, seek slider, ► Тоглуулах / Зогсоох, Хурд slider.
- **Reset:** |◄ товч эсвэл туршилтг дахин сонгох (хөдөлгүүр туршилтыг эхнээс нь дахин тооцоолно).
- **? Ойлголтууд** товч: ойлголтын товч тайлбар (Max, Need, SAFE, UNSAFE, DEADLOCK).

6.2 Юуг ажиглах вэ

- **Зүүн панел – Resource-allocation graph.** Зүүн талд процессууд (P0…), баруун талд нөөцийн хайрцаг (R0…). Хайрцаг дотор нэгж бүр нэг цэг: өнгөт цэг = процесст олгогдсон (өнгө = эзэн), цагаан цэг = чөлөөтэй. Цэнхэр сум = Allocation, бүдүүн улбар шар = одоогийн хүсэлт, улаан = хүлээж буй хүсэлт, саарал = Need.
- **Баруун дээд – матриц.** Allocation / Max / Need, Total, Available, Work, Request. Мөрийн өнгө процессын төлвийг заана.
- **Баруун доод – псевдокод.** Одоогийн алхамд харгалзах мөр цэнхэрээр тодорно (Request / Safety / Detection гэсэн гурван блок).
- **Доод – тайлбар.** Алхмын гарчиг, тайлбар, ✔/✘ шалгалтууд; хамгийн доор тоолуур (шалгалт, зөвшөөрсөн, хүлээлгэсэн, татгалзсан, дууссан) ба Safe sequence.
- **Баруун дээд badge.** SAFE (ногоон) / UNSAFE (улбар шар) / DEADLOCK (улаан) / Төлөв шалгагдаагүй (саарал).

6.3 туршилтуд

№	туршилт	Banker	Юуг харуулах
1	Сурах бичгийн жишээ: хүсэлтүүд	Асаалттай	P1 зөвшөөрөгдөнө; P4 нөөц хүрэхгүй тул хүлээнэ; P0 нөөц байгаа ч UNSAFE тул татгалзана
2	Нөөц байгаа ч өгөхгүй	Асаалттай	1 нөөц, 3 процесс: Banker яагаад татгалзаж байгааг харна
3	Банкиргүй: DEADLOCK	Унтраалттай	2-той ижил систем, Banker-гүй бол түгжрэл гарна
4	Deadlock илрүүлэлт: түгжрэл ҮГҮЙ	Унтраалттай	Detection бүх процесс дуусаж чадна гэж дүгнэнэ
5	Deadlock илрүүлэлт: түгжрэл БАЙНА	Унтраалттай	Нэг нэгж нөөцийн өөрчлөлт P1–P4-ийг түгжрэлд оруулна
6	4 төрлийн нөөц, дасгал 8.3	Асаалттай	P1-ийн нэг хүсэлтийн шийдвэр
7	Дасгал 8.9 (а): Available=(0,3,0,1)	Асаалттай	Эхний төлөв аюулгүй эсэх
8	Дасгал 8.9 (б): Available=(1,0,0,2)	Асаалттай	7-той ижил хүснэгт, өөр Available
9	Дасгал 8.28: хүсэлтүүд	Асаалттай	Эхний хүсэлт төлвийг өөрчлөх тул хоёр дахь хүсэлтийн хариу нөлөөлөгдөнө
10	Том систем: 10 процесс, 4 нөөц	Асаалттай	Олон хүсэлт, зарим нь хүлээнэ; графын уншигдах чанар

7. Туршилтын арга

Зүйл	Тодорхойлолт
Орчин	Ubuntu 24.04, OpenJDK 21.0.12 (javac 21.0.12.1), Xvfb виртуал дэлгэц (screenshot авахад). Нэмэлт сангүй
Өгөгдөл	`ScenarioLibrary`-ийн 10 туршилт (сурах бичгийн жишээ ба дасгал), туршилтын кодод тодорхойлсон өөр өгөгдөл (E1–E4), санамсаргүй төлөв (fuzz: seed=42, n=2…5, m=1…3, 3000 төлөв)
Автомат тест	`tests/AlgorithmTests.java` – гадаад сангүй, `./test.sh` эсвэл `test.bat`-аар ажиллана; гаралт `tests/results.txt`
Бие даасан шалгагч	`validSequence`: өгөгдсөн дараалал Safe sequence мөн эсэхийг алгоритмаас үл хамааран шалгана. `bruteSafe`: бүх $n!$ permutation-ыг туршина
Хүлээгдсэн үр дүн	Сурах бичгийн жишээнүүдийн хариу (SAFE/UNSAFE, олгох/хүлээх/татгалзах), инвариант: 'зөвшөөрсөн хүсэлтийн дараа систем SAFE', 'татгалзсан хүсэлт төлвийг өөрчлөхгүй'
Дахин шалгах	Repository clone → `./test.sh`. Fuzz тест тогтмол seed-тэй тул үр дүн давтагдана

8. Үр дүн

8.1 Автомат тестийн үр дүн

Нийт 29 тест, 29 тэнцсэн, 0 унасан (бодит гаралт: `tests/results.txt`).

ID	Оролт / нөхцөл	Хүлээгдсэн	Бодит	Үр дүн
T1a–c	Сурах бичгийн эхний төлөв	SAFE; олдсон дараалал ба ⟨P1,P3,P4,P0,P2⟩ хоёулаа хүчинтэй	SAFE, ⟨P1,P3,P0,P2,P4⟩	PASS
T2, T2b	P1 хүснэ (1,0,2)	GRANTED; Available=(2,3,0)	GRANTED; (2,3,0)	PASS
T3	P1-ийн дараа P4 хүснэ (3,3,0)	MUST_WAIT (Available=(2,3,0))	MUST_WAIT	PASS
T4, T4b	P0 хүснэ (0,2,0), нөөц байгаа	UNSAFE тул татгалзах; төлөв өөрчлөгдөхгүй	UNSAFE; (2,3,0) хэвээр	PASS
T4c	Ижил (0,2,0) эхний төлөвөөс	GRANTED (төлвөөс хамаарна)	GRANTED	PASS
T5	P1 хүснэ (9,0,0) > Need	EXCEEDS_NEED	EXCEEDS_NEED	PASS
T6a–c	ID=9; (-1,0,0); урт 2 вектор	INVALID	INVALID ×3	PASS
T7a–b	Allocation > Max; Available < 0	IllegalArgumentException	Монгол мессежтэй exception ×2	PASS
T8a–b	Тэг хүсэлт; 1×1 систем Need=0	GRANTED; SAFE	GRANTED; ⟨P0⟩	PASS
T9a	Дасгал 8.9(а) Available=(0,3,0,1)	Үр дүнг бүртгэх	UNSAFE, олдсон нь ⟨P2,P1,P3⟩ (3/5)	PASS
T9b	Дасгал 8.9(б) Available=(1,0,0,2)	Үр дүнг бүртгэх	SAFE ⟨P1,P2,P0,P3,P4⟩	PASS
T10a–c	туршилт 2, 3, 4, 5 эцсийн төлөв	DONE, DEADLOCK, DONE, DEADLOCK	тохирсон	PASS
E1	Өөр өгөгдөл: Need-ээс хэтэрсэн хүсэлт	ERROR алхам, Татгалзсан ≥ 1	ERROR, cDenied=1	PASS
E2	Бүх Need=0	DONE, 2/2	DONE, 2/2	PASS
E3	Available=0, Banker асаалттай, бодит түгжрэл	DEADLOCK	DEADLOCK	PASS
E4	Max > Total (5 > 2)	DEADLOCK	DEADLOCK	PASS
F1	3000 санамсаргүй төлөв	checkSafety = брут-форс	зөрүү = 0 (SAFE: 1927)	PASS
F2	Fuzz: олдсон дараалал бүр	бие даасан шалгагчаар хүчинтэй	буруу = 0	PASS
F3	Fuzz: зөвшөөрсөн хүсэлт бүр	дараа нь SAFE хэвээр	716 зөвшөөрөл, зөрчил = 0	PASS

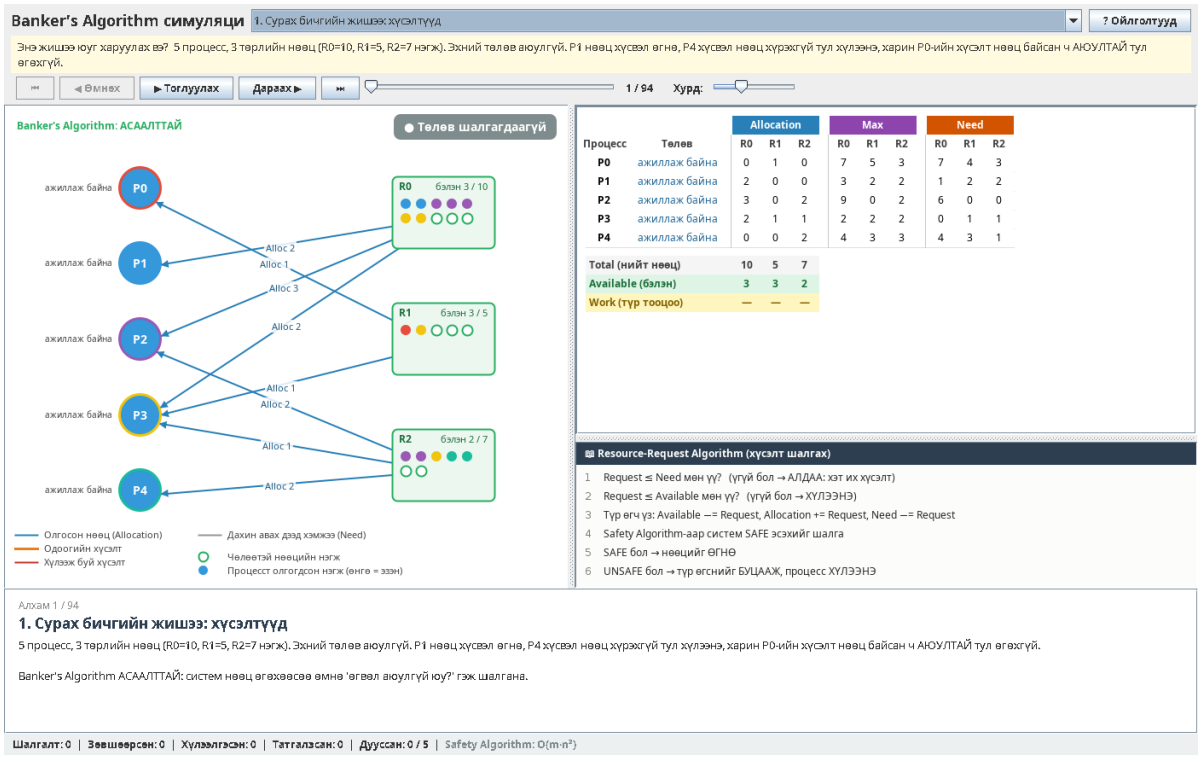
Тестийн явцад гарсан засвар. Анхны тест T1, T3, T4 унасан. Шалтгаан нь кодын алдаа биш, миний бичсэн тестийн алдаа байсан: (1) T1 нь сурах бичгийн ганц дарааллыг шаардсан (Safe sequence олон байж болно); (2) T3, T4 нь хүсэлтүүдийг сурах бичгийн дарааллаар (P1-ийн дараа P4, P0) биш эхний төлвөөс туршсан. Алгоритмыг гараар бодож (3.7 хэсэг) тестийг засаж, T4с нь 'ижил хүсэлт өөр төлөвт өөр хариу өгдөг' гэдгийг нотлох нэмэлт тест болсон.

8.2 туршилтйн үр дүн

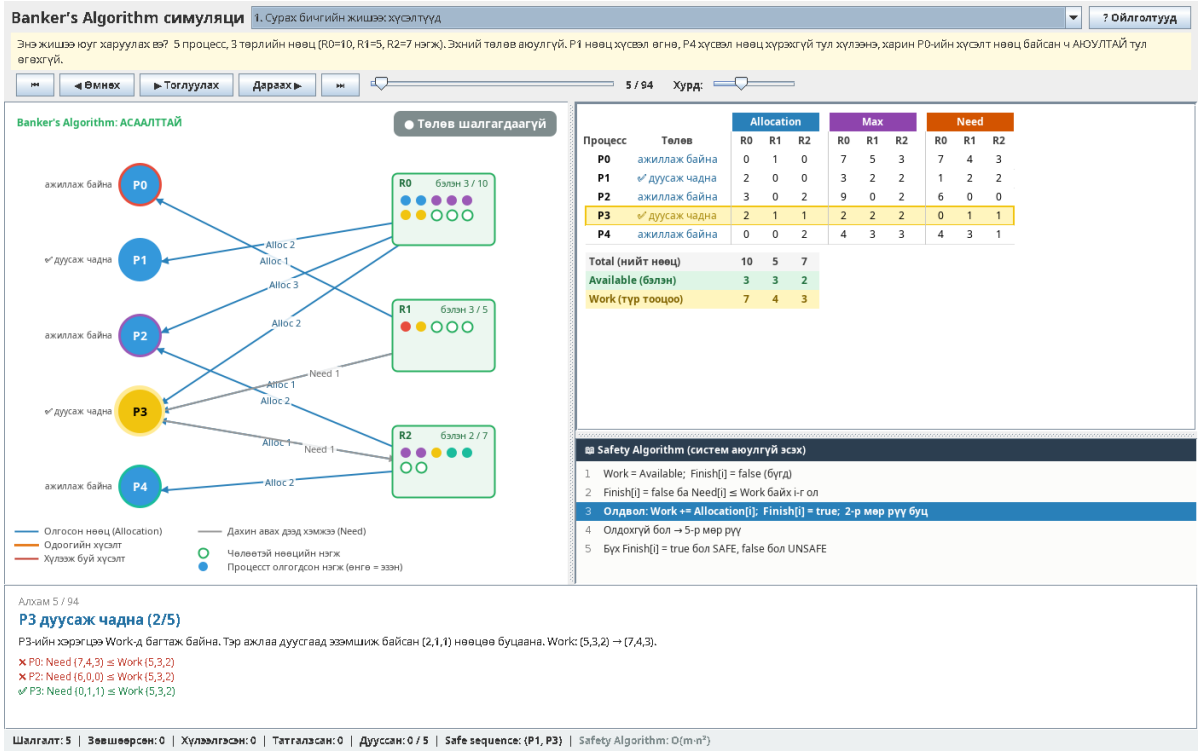
№	Алхам	Төгсгөлийн төлөв	Зөвшөөрсөн	Хүлээлгэсэн	Татгалзсан	Дууссан
1	94	DONE	8	1	2	5/5
2	63	DONE	4	0	3	3/3
3	12	**DEADLOCK**	1	0	0	1/3
4	29	DONE	3	0	0	5/5
5	7	**DEADLOCK**	0	0	0	1/5
6	46	DONE	5	0	0	5/5
7	16	**DEADLOCK**	0	0	0	3/5
8	37	DONE	5	0	0	5/5
9	66	DONE	7	1	0	5/5
10	135	DONE	12	2	0	10/10

туршилт 2 ба 3 нь яг ижил систем бөгөөд Banker асаалттай үед түгжрэл гарахгүй (DONE, 3/3), унтраалттай үед DEADLOCK (1/3 дууссан) болж байгаа нь алгоритмын ач холбогдлыг шууд харуулна. туршилт 7-д Banker асаалттай боловч хүсэлтгүй, эхний төлөв UNSAFE тул үлдсэн процессууд түгжрэлд орно (3/5 дууссан).

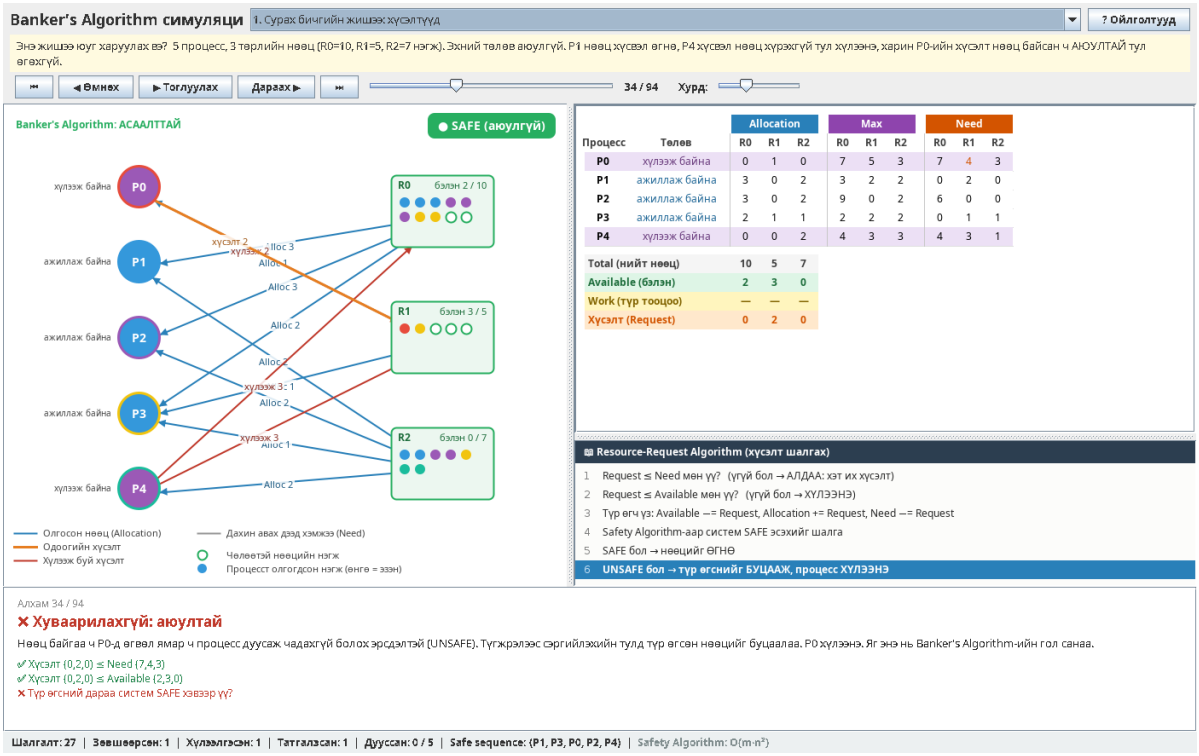
8.3 Screenshot (Xvfb дээр бодитоор ажиллуулж авсан)



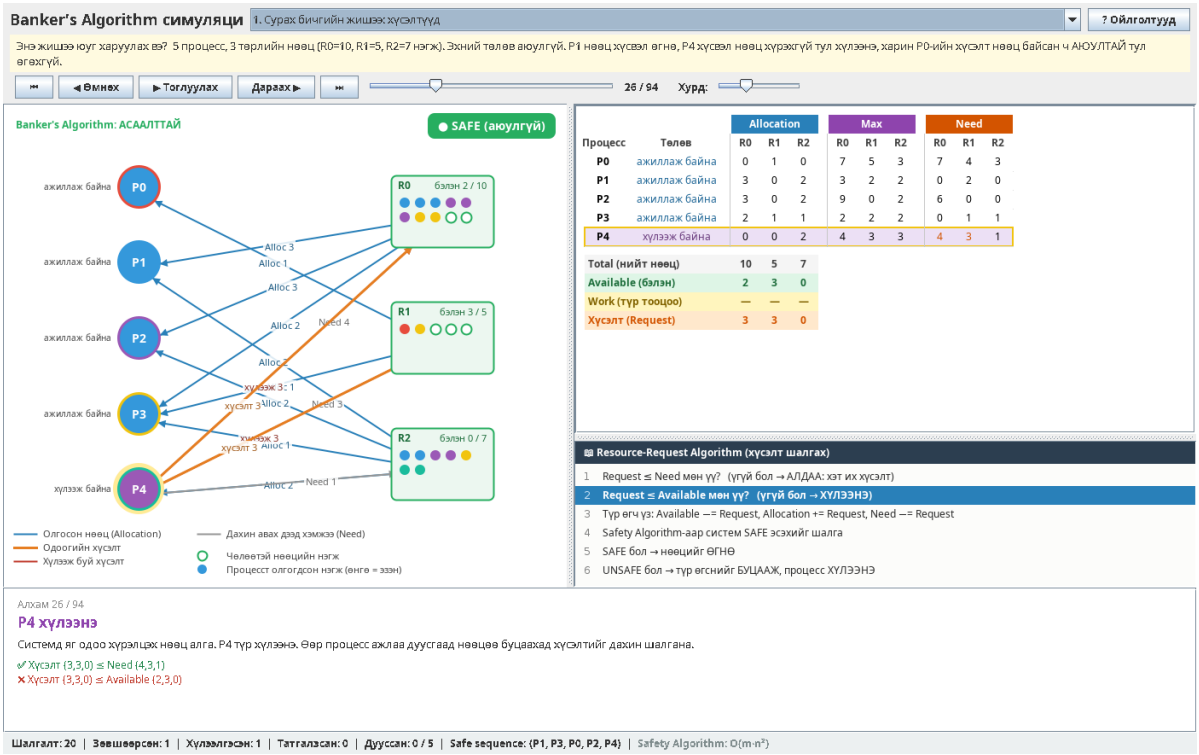
Зураг 8.1. туршилт 1-ийн эхлэл: Banker АСААЛТТАЙ, процесс ба нөөцийн граф, матриц, псевдокод, тайлбар.



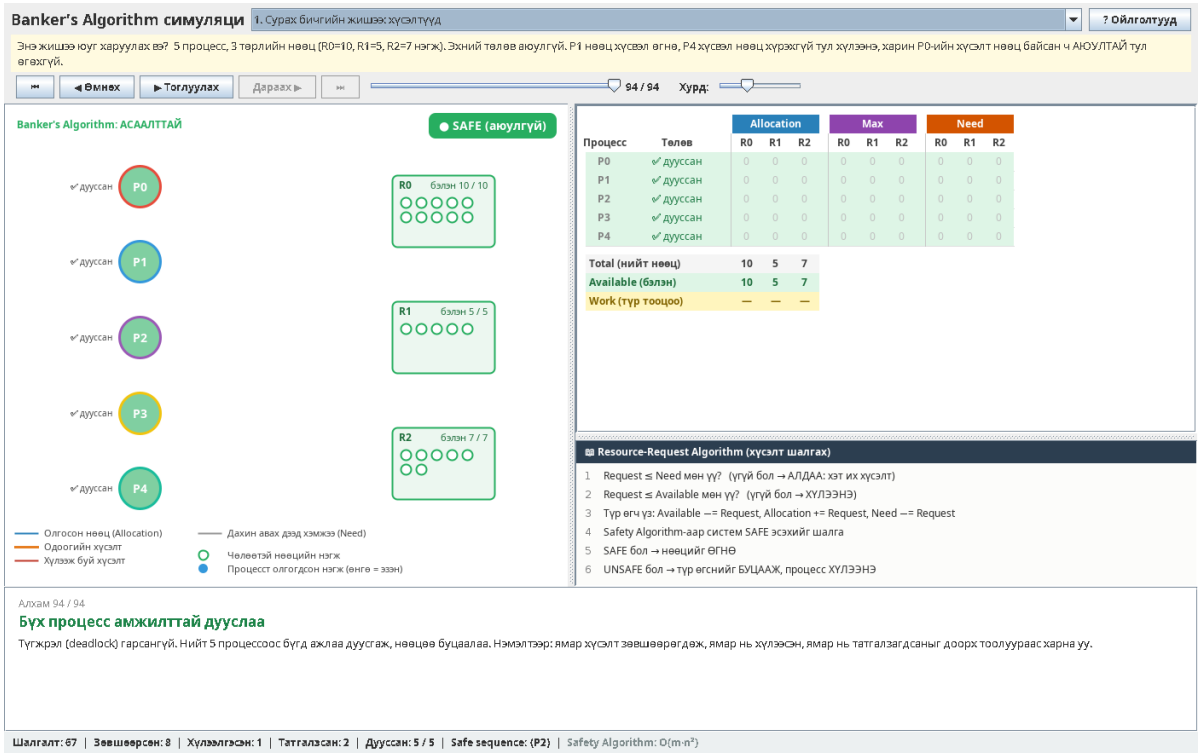
Зураг 8.2. Safety Algorithm ажиллаж байна (алхам 5/94): P3 дуусаж чадна, Work (5,3,2) → (7,4,3). P0, P2-ийн Need Work-оос их тул ✘.



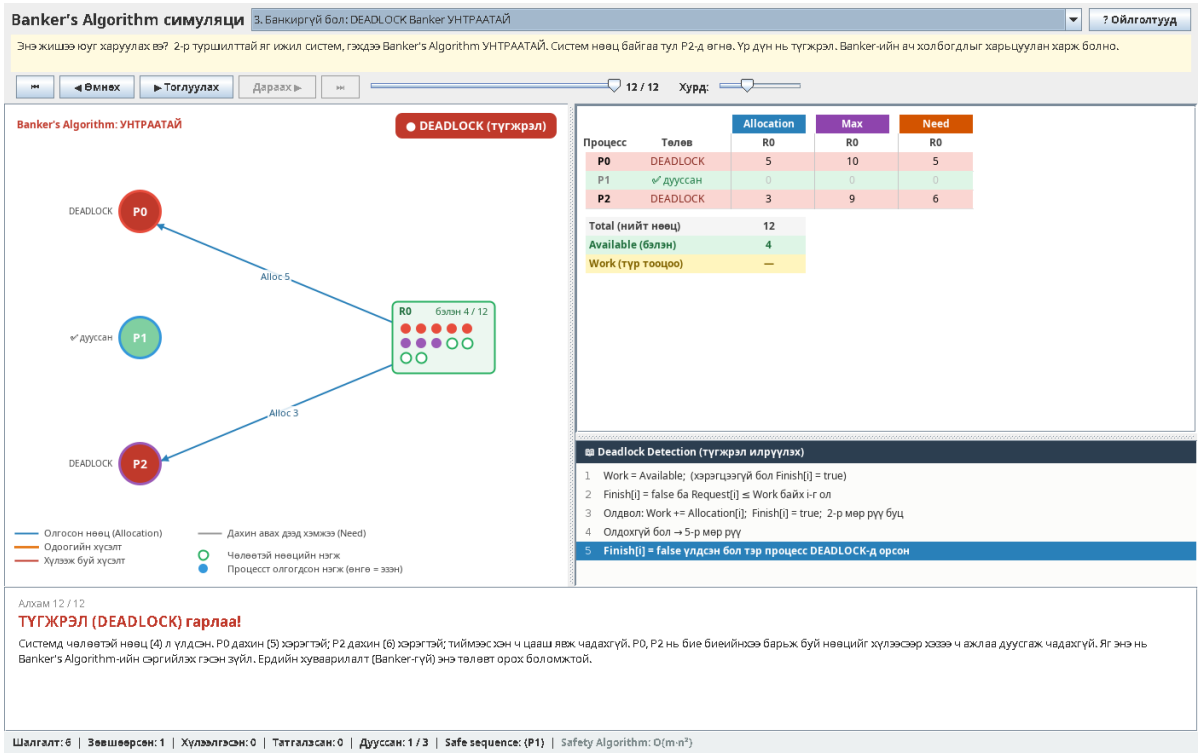
Зураг 8.3. P0-ийн (0,2,0) хүсэлт: Need, Available шалгалт тэнцсэн ч түр олгосны дараа UNSAFE тул татгалзав. Псевдокодын 6-р мөр тодорсон.



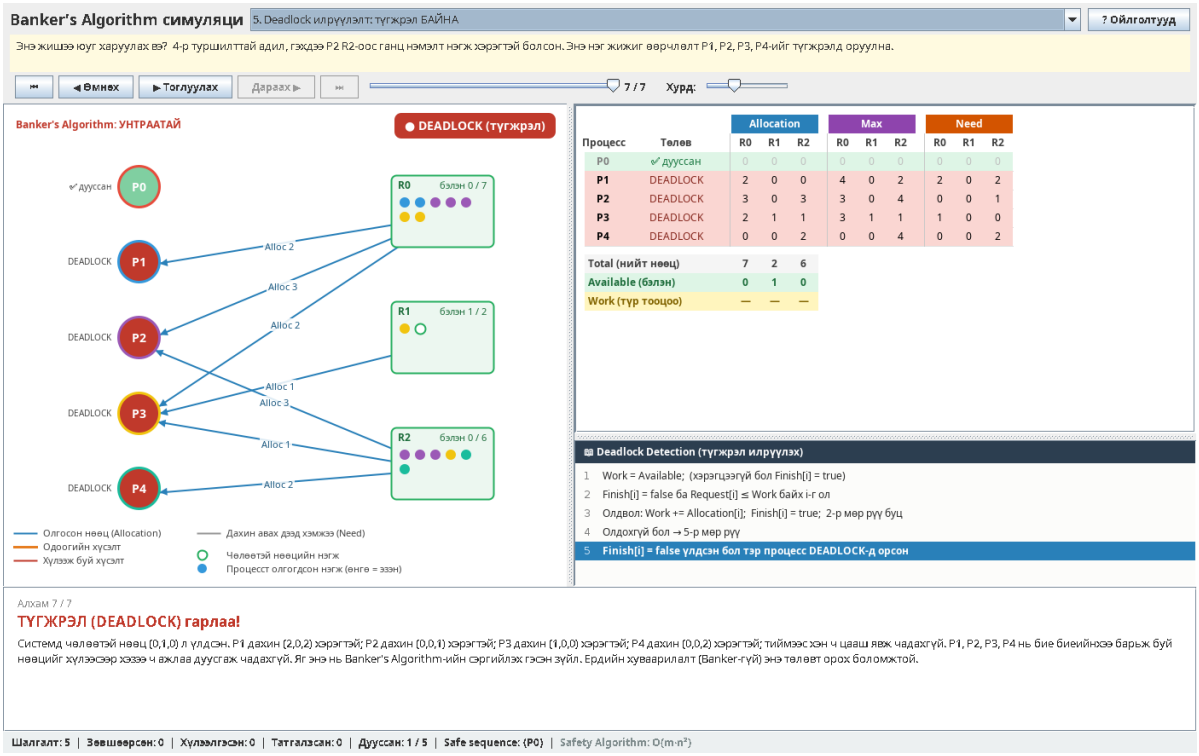
Зураг 8.4. P4-ийн (3,3,0) хүсэлт Available (2,3,0)-оос их тул хүлээнэ (улаан сум = хүлээж буй хүсэлт).



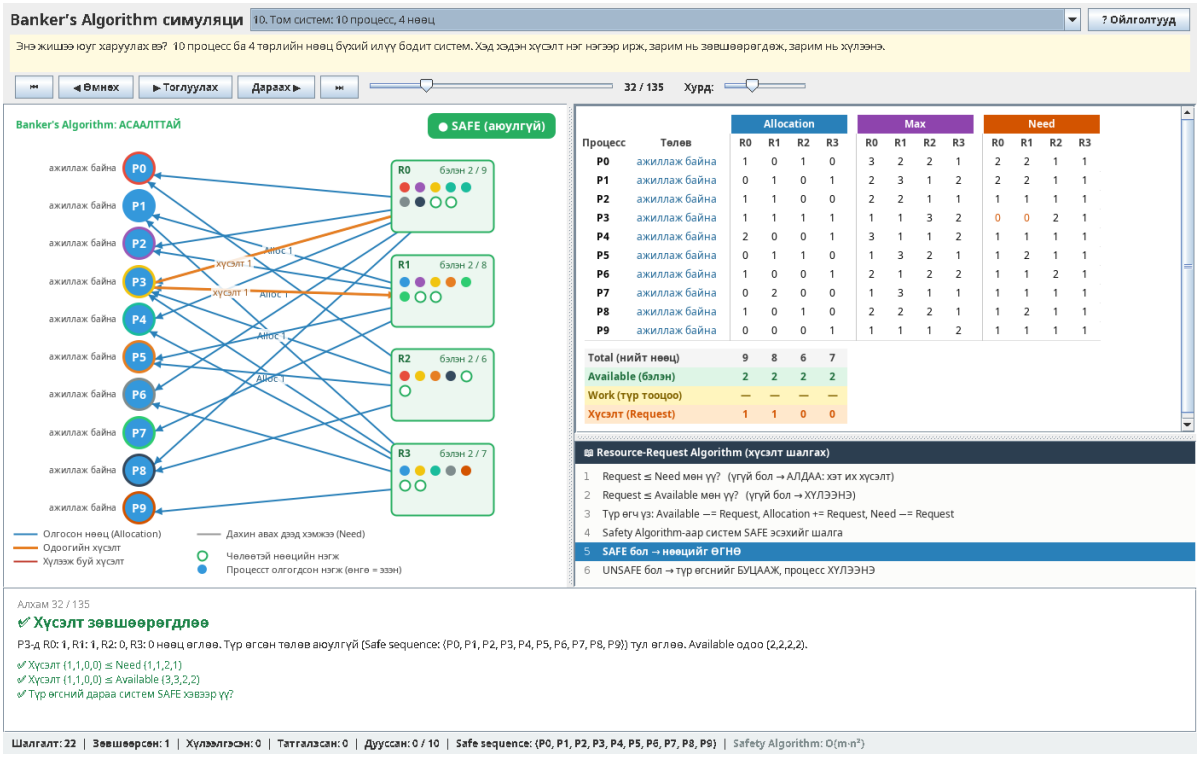
Зураг 8.5. туршилт 1-ийн төгсгөл: 5/5 процесс дууссан, нөөц бүгд чөлөөтэй.



Зураг 8.6. туршилт 3 (Banker УНТРААТАЙ): P0 ба P2 түгжрэлд орсон; чөлөөтэй нөөц 4, харин Need 5 ба 6.



Зураг 8.7. туршилт 5: Deadlock Detection P1–P4-ийг түгжрэлтэй гэж тодорхойлсон.



Зураг 8.8. туршилт 10: 10 процесс, 4 нөөцтэй том систем.

9. Хэлэлцүүлэг

9.1 Үр дүнгийн тайлбар

- Тест T3, T4, T4c хамтдаа Banker-ийн гол санааг баталгаажуулна: нөөц байгаа эсэх нь хангалттай нөхцөл биш, ижил хүсэлтийн хариу *системийн төлвөөс* хамаарна.
- Fuzz F1 нь гүйцэтгэлийн хувьд хурдан Safety Algorithm-ийн дүгнэлт $n!$ permutation-ы брут-форстой 3000 удаа давхцаныг харуулсан. Энэ нь жижиг хэмжээс ($n \leq 5$, $m \leq 3$) дээрх баталгаа бөгөөд том хэмжээст тохиолдлыг нотлохгүй.
- F3 нь Banker-ийн гол инвариант (’зөвшөөрсөн хүсэлт системийг SAFE-ээс гаргахгүй’) хэрэгжилтэд биелж байгааг харуулсан.
- туршилт 7 ба 8 нь ижил хүснэгтэд зөвхөн Available-ийг өөрчлөхөд SAFE/UNSAFE дүгнэлт эсрэгээр болохыг харуулна.

9.2 Хязгаарлалт ба мэдэгдэж буй асуудлууд

- **Хэрэглэгчийн оролт.** UI дээрээс Max, Allocation, Available, хүсэлтийг гараар оруулах, туршилт засварлах боломжгүй. Өөр өгөгдлөөр турших бол `ScenarioLibrary.java`-г засаж дахин compile хийнэ. Иймд удирдамжийн ’оролт, нөхцөл, параметрийг өөрчилж дахин турших’ ба ’буруу оролтыг шалгаж ойлгомжтой мэдээлэх’ шаардлагыг UI түвшинд зөвхөн хэсэгчлэн хангана. Баталгаажуулалт (`SystemState.validate`, `requestResources`) хөдөлгүүрийн түвшинд бий, тестээр шалгагдсан (T5–T7, E1) боловч UI-тай холбогдоогүй.
- **Алгоритмын давхар хэрэгжилт.** `SimulationEngine.processRequest` нь `BankerAlgorithm.requestR` ийг дуудахгүй, өөрийн логикоор (тайлбар текст үүсгэхийн тулд) давхар хэрэгжүүлсэн. Fuzz тест зөвхөн `BankerAlgorithm`-ийг шалгасан; хөдөлгүүрийн олгох/татгалзах шийдвэрийг түүнтэй шууд харьцуулсан тест одоогоор байхгүй (зөвхөн туршилтйн эцсийн төлвийг шалгасан). Хоёр хэрэгжилт хойтон зөрөх эрсдэлтэй.
- **Detection-ий Request.** Request вектор нь Need-тэй тэнцүү гэж авсан (3.5). Сурах бичгийн 8.6.2 жишээ тусдаа Request матрицтай; туршилт 4, 5-д Need нь тэр жишээний Request-тэй давхцахаар сонгогдсон тул жишээнүүд таарна.
- **Release ба thread.** Тусдаа нөөц чөлөөлөх хүсэлт, олон thread-тэй үйлчлүүлэгч хэрэгжээгүй (2-р бүлэг).

10. Дүгнэлт

Төсөл нь Banker’s Algorithm-ийн Safety, Resource-Request, Deadlock Detection гурвыг Java Swing дээр хэрэгжүүлж, Resource-allocation graph бүхий алхам алхмаар тоглуулдаг симулятор болгосон. 10 туршилт, 29 автомат тест (үүнд 3000 төлөвт брут-форстой харьцуулсан fuzz тест) амжилттай тэнцэж, Banker-ийн гол санаа болох ’нөөц байгаа ч аюултай тул олгохгүй’ ба ’Banker-гүй бол түгжрэл’ хоёрыг нэг системээр харьцуулан харуулсан.

Хамгийн чухал дутагдал нь хэрэглэгч UI-аас өөрийн өгөгдлөө оруулж чаддаггүй явдал. Дараагийн алхамд туршилт засварлах форм (Available/Max/Allocation/хүсэлт), оролтын алдааг дэлгэцэд мэдээлэх, хөдөлгүүрийг `BankerAlgorithm.requestResources`-тэй нэгтгэх, release хүсэлт нэмэх нь тус болно.

Ашигласан материал

- A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*. Chapter 8: Deadlocks (Banker’s Algorithm, Deadlock Detection, Programming Projects). Хэвлэлийн дугаарыг баг

нөхнө.

- E. W. Dijkstra. Banker's algorithm-ийн анхны санал (EWD108), 1965.
- Oracle. Java SE Documentation: `javax.swing`, `java.awt`.
- Төслийн эх код: `https://github.com/Tsendsvren/csm302_biydaalt`

Хавсралт А. Ажлын төлөвлөгөө

Талбар	Мэдээлэл
Хуваарилалтын код	II-4-13
Төсөл	Banker's Algorithm (Resource graph UI)
Repository	`https://github.com/Tsendsvren/csm302_biydaalt`
Хэл/сан/орчин	Java 17+, Swing/AWT, гадаад сангүй, `javac`
Үндсэн дүрслэл	Resource-allocation graph (процесс ба нөөцийн граф)

Гишүүн		Хариуцах ажил	Commit/file
B242270128	Ц.Цэндсүрэн	Алгоритм, симуляцийн хөдөлгүүр, UI (git түүхээр)	`src/main/java/com/os/*`
B231940023	.....	.....	.....
B241910004	.....	.....	.....
B241940034	.....	.....	.....
B242270041	.....	.....	.....

Тест	Оролт/нөхцөл	Хүлээгдсэн үр дүн
Сурах бичгийн жишээ	туршилт 1, Available=(3,3,2)	P1 олгоно; P4 хүлээнэ; P0 татгалзана
Banker-ийн ач холбогдол	туршилт 2 ба 3	2: DONE 3/3; 3: DEADLOCK 1/3
Detection	туршилт 4 ба 5	4: түгжрэлгүй; 5: P1–P4 түгжрэл
Захын / буруу оролт	T5–T8, E1–E4	EXCEEDS_NEED, INVALID, exception, DEADLOCK/DONE
Fuzz	3000 санамсаргүй төлөв	брут-форстой 100% давхцал